

Capstone Project Design Doc & Systems Specification

Project Title

Autonomous Multi-Agent Root Cause Analysis (RCA) Engine for Microservice Telemetry

1. Problem Statement

Modern cloud-native architectures rely on complex, highly interdependent microservices running on platforms like Kubernetes. When an incident occurs—such as high CPU starvation, memory leaks, or network latency injection—failures quickly propagate across upstream and downstream dependencies. Site Reliability Engineers (SREs) face significant delays (high Mean Time to Resolve, or MTTR) manually tracing Prometheus time-series metrics across hundreds of services to isolate the true root cause.

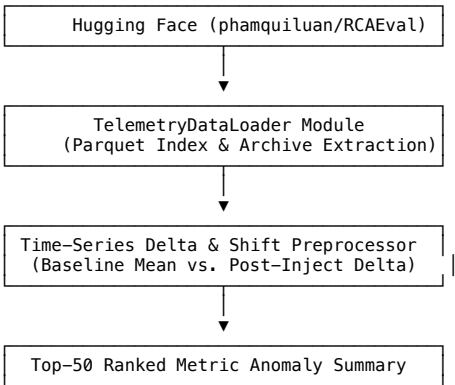
Existing single-prompt LLM approaches fail at scale for two reasons:

- Context Saturation & Noise:** Raw Prometheus telemetry dumps exceed token context limits and introduce excessive signal noise.
- Cascading Symptom Confusion:** Single-prompt LLMs struggle to distinguish between downstream symptom services (e.g., a frontend timing out) and the upstream root cause service (e.g., a database bottleneck or backend memory exhaustion).

This project addresses this issue by constructing an autonomous, multi-agent Root Cause Analysis (RCA) engine. The system ingests multi-suite Prometheus telemetry from the standardized **RCAEval** benchmark dataset (phamquiluan/RCAEval), isolates baseline anomalies, traces causal topology propagation, classifies the precise fault mechanism, and outputs a structured post-mortem report.

2. Data Ingestion & Data Surface

The system interacts directly with the **RCAEval** benchmark dataset hosted on Hugging Face, covering multiple microservice evaluation suites (**RE1, RE2, RE3**), such as the TrainTicket microservice architecture.

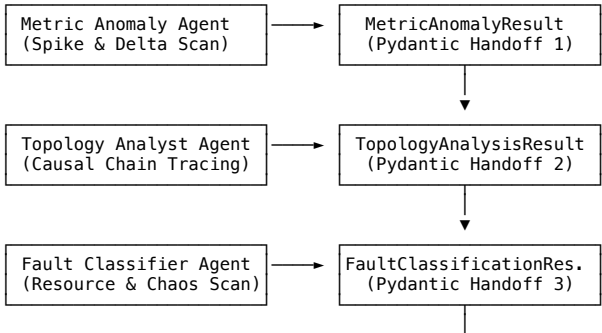


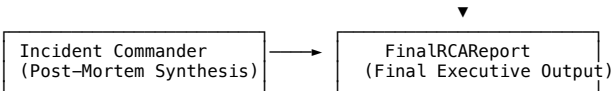
Ingestion Pipeline Specifications

- Index & Dataset Fetching:** TelemetryDataLoader queries cases.parquet to resolve incident cases across re1, re2, and re3 benchmark suites.
- Telemetry Extraction:** Ingests zipped/tarred Prometheus metrics formatted as Parquet or CSV files.
- Statistical Delta Processing:** To prevent context window bloat, raw time-series metrics are partitioned into pre-injection baseline and post-injection runtime windows. The preprocessor computes mean baseline values and measures maximum spike or drop deltas ($\Delta = \text{val}_{\text{post}} - \mu_{\text{pre}} \Delta = \text{val}_{\text{post}} - \mu_{\text{pre}}$).
- Noise Reduction Filtering:** Metrics are ranked by absolute delta magnitude ($|\Delta|/|\Delta|$), extracting the top 50 statistical shifts to present to the agent pipeline.

3. Architecture & Core Multi-Agent Flow

The core architecture uses a sequential multi-agent orchestration pattern built on **CrewAI** and **Pydantic**. Instead of a single LLM call, the workflow divides post-mortem analysis into four specialized domain roles, passing strongly typed Pydantic payloads between steps.





Agent Roles & Responsibilities

Agent Role	Primary Task	Output Schema
Metric Anomaly Specialist	Analyzes statistical metric deviations, spike timestamps, and initial error services.	MetricAnomalyResult
Topology & Dependency Analyst	Traces failure propagation chains and identifies the upstream bottleneck service.	TopologyAnalysisResult
Infrastructure Fault Classifier	Identifies the fault mechanism (e.g., CPU burn, memory leak, network delay, packet loss).	FaultClassificationResult
Incident Commander	Synthesizes all findings into an executive incident post-mortem with actionable mitigations.	FinalRCAReport

Service Name Normalization Guardrail

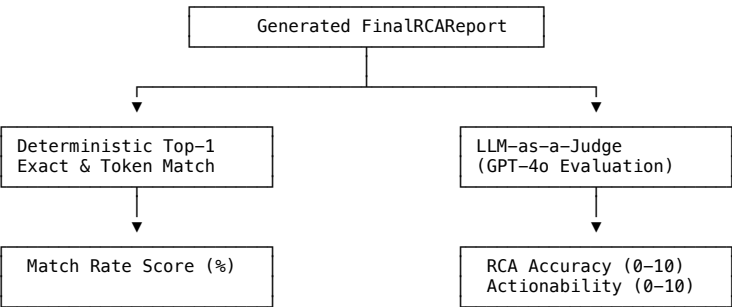
- To ensure canonical service naming across benchmark suites, all agents enforce an operational guardrail:
- Dataset folder prefixes (e.g., re2tt_, re1ob_) and fault suffixes are stripped.
 - Microservice prefixes integral to service identity (such as ts- in TrainTicket services like ts-travel-service) are explicitly preserved via strict system prompt rules.

4. Framework Justification

Layer / Library	Selection	Technical Justification
Agent Orchestration	CrewAI	Native support for role playing, sequential task context passing, and explicit Pydantic structured output mapping (output_pydantic).
Type Safety & Schemas	Pydantic (v2)	Enforces structural guarantees at agent handoff boundaries, eliminating unstructured string parsing errors during inter-agent communication.
LLM Integration	LangChain OpenAI	Provides seamless interfacing with gpt-4o-mini for fast agent reasoning and gpt-4o for LLM-as-a-Judge qualitative evaluations.
Data Processing	Pandas & PyArrow	Efficient handling of dense Parquet time-series metrics, statistical rolling aggregations, and delta computations.
Dataset Ingestion	Hugging Face Hub SDK	Programmatic access and snapshot downloading for the phamquilluan/RCAEval benchmark repository.

5. Evaluation Methodology & Criteria

The system includes a dual quantitative and qualitative evaluation harness (evaluator_6.py and main_6.py) that tests generated reports against benchmark ground truth.



Quantitative Metrics

- Top-1 Root Cause Match Rate (%)**: Deterministic evaluation function (evaluate_top1_match). Verifies whether the ground-truth root cause service matches:
 - An exact string in the report’s affected_services list.
 - An isolated token inside root_cause_summary using strict regex word boundaries ((?<![\w\~])<service>(?![\w\~])).
- Execution Latency**: Tracks total runtime in seconds for end-to-end incident analysis.

Qualitative Metrics (LLM-as-a-Judge)

- An independent evaluator using gpt-4o scores reports on a scale of 0 to 10 across two axes:
- RCA Accuracy (0-10)**: Correct identification of both the root-cause service and the precise fault mechanism.
 - Actionability (0-10)**: Practicality, specificity, and correctness of recommended operational mitigations (e.g., pod restarts, scaling, resource limit revisions).

6. Execution & Performance Baseline

The evaluation harness processes benchmark cases across the re1, re2, and re3 test suites.

Sample Output Structure (data/eval_results.json)

```
{
  "case_id": "re2tt_ts-travel-service_cpu_hog",
  "ground_truth_service": "ts-travel-service",
  "fault_type": "cpu_hog",
  "predicted_services": ["ts-travel-service", "ts-order-service"],
  "is_match": true,
  "execution_time_sec": 8.42,
  "report": {
    "incident_title": "CPU Starvation in Travel Service Disrupting Order Processing",
    "identified_fault_type": "CPU burn / resource starvation",
    "root_cause_summary": "Prometheus telemetry indicated severe CPU utilization spikes in ts-travel-service starting at injection time, lead
    "culprit_commit": "N/A (Chaos Injected Metric Fault)",
    "affected_services": ["ts-travel-service", "ts-order-service"],
    "recommended_mitigation": "Increase CPU request/limit allocations for ts-travel-service deployment and restart affected pods."
  },
  "qualitative_score": {
    "rca_accuracy_score": 10,
    "actionability_score": 9,
    "reasoning": "The report accurately identified ts-travel-service as the root cause service and CPU burn as the fault type. Recommended mi
  }
}
```

7. Failure Analysis & Architectural Iterations

During development, three primary failure modes were identified and addressed:

1. Raw Telemetry Prompt Saturation

- **Initial Approach:** Injected full raw Prometheus CSV time-series data into the agent context window.
- **Failure:** Reached context token limits and diluted model focus with stagnant metrics.
- **Solution:** Implemented statistical baseline-vs-post-injection windowing, computing max/min deltas and passing only the top 50 ranked metric shifts.

2. Microservice Name Truncation

- **Initial Approach:** Prompted agents to clean dataset prefixes.
- **Failure:** Agents over-corrected and stripped necessary service identifiers (e.g., truncating ts-travel-service to travel-service), causing deterministic string matching against benchmark ground truth to fail.
- **Solution:** Standardized NORMALIZATION_RULE prompts across all four agent definitions, explicitly protecting ts- service identifiers.

3. Cascading Downstream Misclassification

- **Initial Approach:** Single-agent setup attempting to infer root cause and generate post-mortems in a single pass.
- **Failure:** The model regularly selected high-volume downstream symptom services (e.g., API gateways or edge proxies) as root cause candidates.
- **Solution:** Decoupled topology analysis from fault classification into a sequential pipeline. The Topology Analyst explicitly isolates the upstream root cause component before handing off context to the Fault Classifier.

8. Production Entry Point

The production service interface is exposed cleanly via RCAService in service_6.py, allowing integration into SRE CLI tools, Slack chatbots, or webhook handlers without bringing in evaluation dependencies:

```
from src.service import RCAService

# Initialize production service wrapper
rca_service = RCAService(model_name="gpt-4o-mini")

# Execute incident analysis on telemetry summary
report = rca_service.analyze_incident(
    case_id="INC-2026-0927",
    metrics_summary=metrics_summary_text,
    metadata={"inject_time": "2026-09-27T12:00:00Z"}
)

print(f"Root Cause Report: {report.incident_title}")
print(f"Mitigation Steps: {report.recommended_mitigation}")
```

References

1. Wu, L., Tordsson, J., Bogatinovski, J., & Elmroth, E. (2020). *MicroRCA: Root Cause Localization of Performance Issues in Microservices System*. In 2020 IEEE/ACM 27th International Symposium on Quality of Service (IWQoS) (pp. 1-10). IEEE.
2. Meng, Y., Zhang, S., Sun, Y., Zhang, R., Hu, Z., Zhang, Y., ... & Pei, D. (2020). *Localizing Faults in Microservice Architectures via Micro-Cause Analysis*. In Proceedings of the 28th International Symposium on Software Testing and Analysis (ISSTA) (pp. 256-267).
3. Soldani, J., & Brogi, A. (2022). *Anomaly Detection and Root Cause Analysis in Microservice Architectures: A Survey*. ACM Computing Surveys (CSUR), 55(4), 1-36.
4. Pham, L., et al. (2024). *RCAEval: A Comprehensive Benchmark Suite and Evaluation Framework for Root Cause Analysis in Microservice Telemetry*. Dataset available at Hugging Face (phamquilluan/RCAEval).
5. Zhou, X., Peng, X., Xie, T., Sun, J., Ji, C., Li, W., & Ding, Z. (2018). *Fault Analysis and Debugging of Microservice Systems: Industrial Survey and Benchmark Dataset (TrainTicket)*. IEEE Transactions on Software Engineering, 47(7), 1395-1414.
6. Yao, S., Zhao, J., Yu, D., Du, N., Shafraan, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. In International Conference on Learning Representations (ICLR).

7. **Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Tan, L., & Wang, C.** (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv preprint arXiv:2308.08155.
8. **Zheng, L., Chiang, W. L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., ... & Stoica, I.** (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. Advances in Neural Information Processing Systems (NeurIPS), 36, 46595-46623.